Antes de comenzar a realizar los ejercicios sobre flujo de bytes es importante que **leas y comprendas "la lectura en bloques"**

Lectura en bloques con buffer en Java

1) ¿Qué significa "leer en bloques"?

- En lugar de leer **byte a byte** (lento), leemos **muchos bytes de una vez** en un byte[] (un **buffer**).
- Menos llamadas al SO ⇒ **más rendimiento**

```
byte[] buffer = new byte[8192]; // 8 KiB
int n;
while ((n = in.read(buffer)) != -1) {
   // procesar SOLO buffer[0..n-1]
}
```

2) Dos "buffers" diferentes

1. **Tu buffer de trabajo**: el byte[] que pasas a read(buffer). Lo creas **una vez** y lo **reutilizas**.
2. **Buffer interno** de clases como BufferedInputStream/BufferedOutputStream: se crea al construirlos y optimiza las lecturas/escrituras desde/hacia el sistema operativo.

Usar ambos juntos es normal y recomendado

```
try (BufferedInputStream in = new BufferedInputStream(new FileInputStream(path.toFile()), 8192)) {
   byte[] buf = new byte[8192];
   int n;
   while ((n = in.read(buf)) != -1) {
      // ...
   }
}
```

3) ¿Siempre hay que hacerlo?

Casi siempre en lectura binaria **secuencial**:

- Copias de archivos, hashing, compresión, cifrado, multimedia…

No siempre:

- Ficheros muy pequeños → Files.readAllBytes(path).

- Cuando usas **APIs de alto nivel** que ya optimizan (p. ej. Files.copy).
- Acceso aleatorio → RandomAccessFile / FileChannel (posicionamiento).

4) ¿Qué tamaño de buffer usar?

- Estándar Java: **8 KiB (8192)**.
- Rango práctico: **4 KiB – 64 KiB**.
- Ficheros muy grandes/SSD rápido: **64–256 KiB** puede ayudar.
- 1 MiB rara vez compensa (más GC/memoria, poca mejora).

Regla: empieza con **8 KiB**; si mides y necesitas, prueba **64 KiB**. Usa múltiplos de **4096** (tamaño de página).

5) ¿Cuándo y dónde se crea el buffer?

- El **buffer interno** se crea en el **constructor** del Buffered* (opcionalmente indicas tamaño).
- Tu byte[] se crea **antes del bucle** y se **reutiliza**; **no** lo crees dentro del bucle.

6) Patrones de uso (plantillas)

6.1 Lectura + escritura (copia) con I/O clásico

```
try (BufferedInputStream in  = new BufferedInputStream(new FileInputStream(src.toFile()), 8192);
   BufferedOutputStream out = new BufferedOutputStream(new FileOutputStream(dst.toFile()),
8192)) {

   byte[] buf = new byte[8192];
   int n;
   while ((n = in.read(buf)) != -1) {
      out.write(buf, 0, n); // ¡ojo: n bytes, no buf.length!
   }
   out.flush();
}
```

6.2 Solo lectura (contar bytes + preview)

```
try (BufferedInputStream in = new BufferedInputStream(new FileInputStream(path.toFile()), 8192)) {
   byte[] buf = new byte[8192];
   long total = 0;
   int n;
   while ((n = in.read(buf)) != -1) {
      total += n;
      // procesar buf[0..n-1]
   }
   System.out.println("Leídos: " + total + " bytes");
}
```

Ejercicios ficheros (flujo de bytes)

Ejercicio paso a paso. Lector bytes (conteo+ vista previa)

Qué debe hacer

1. Recibir por argumento la ruta del fichero.
2. Leerlo como **bytes** (no como texto).
3. Mostrar:
    - Total de bytes leídos.
    - Los **primeros 16 bytes** en **hexadecimal** (o menos si el fichero es pequeño).

Ejemplo de uso

```
javac flujo_bytes\LectorBytesSimple.java
java flujo_bytes.LectorBytesSimple flujo_bytes\listado.pdf
```

Nota: “flujo_bytes” es el nombre del paquete donde está el ejercicio.

Salida (ejemplo):

```
Fichero : C:\...\flujo_bytes\listado.pdf
Tamaño  : 183421 bytes
Preview : 25 50 44 46 2D 31 2E 37 0A 25 E2 E3 CF D3 0A 0A
```

Paso 1 — “Hola, argumentos”

Añade un main que valide que llega **un** argumento (la ruta del fichero) y muestre un mensaje de uso si falta

```
package flujo_bytes;

public class LectorBytes {
  public static void main(String[] args) {
    if (args.length != 1) {
      System.out.println("Uso: java -cp . flujo_bytes.LectorBytes <fichero>");
      return;
    }
    System.out.println("Fichero a leer: " + args[0]);
  }
}
```

Compila y ejecuta desde la carpeta **Ejercicios**:

```
javac flujo_bytes\LectorBytes.java
java -cp . flujo_bytes.LectorBytes flujo_bytes\listado.pdf
```

Paso 2 — Validar la ruta del fichero

Comprueba que **existe**, es **fichero regular** y es **legible**

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

private static void validarFicheroLectura(Path fichero) {
  if (!Files.exists(fichero)) {
    throw new IllegalArgumentException("No existe: " + fichero.toAbsolutePath());
  }
  if (!Files.isRegularFile(fichero)) {
    throw new IllegalArgumentException("No es un fichero regular: " + fichero.toAbsolutePath());
  }
  if (!Files.isReadable(fichero)) {
    throw new IllegalArgumentException("Sin permisos de lectura: " + fichero.toAbsolutePath());
  }
}
```

En main, usa la validación con try/catch

```
Path fichero = Paths.get(args[0]);
try {
  validarFicheroLectura(fichero);
} catch (IllegalArgumentException iae) {
  System.err.println("Parámetros inválidos: " + iae.getMessage());
  return;
}
```

Paso 3 — Leer en bloques (buffer) y contar bytes

- Usaremos **8 KiB** por bloque (8192 bytes).
- Guardaremos los **primeros 16 bytes** para la preview.

```
import java.io.BufferedInputStream;
import java.io.FileInputStream;
import java.io.IOException;

private static final int BUF = 8 * 1024;   // bloque de lectura
private static final int PREVIEW_N = 16;   // bytes de preview

// Dentro de main, tras validar:
long total = 0L;
byte[] buffer = new byte[BUF];
byte[] preview = new byte[PREVIEW_N];
int previewLen = 0;

try (BufferedInputStream in =
      new BufferedInputStream(new FileInputStream(fichero.toFile()), BUF)) {

  int n;
  while ((n = in.read(buffer)) != -1) {    // lee hasta EOF
    total += n;
```

```
      // completar preview con los primeros 16 bytes del fichero
      if (previewLen < PREVIEW_N) {
        int faltan = PREVIEW_N - previewLen;
        int aCopiar = Math.min(faltan, n);
        System.arraycopy(buffer, 0, preview, previewLen, aCopiar);
        previewLen += aCopiar;
      }
    }
} catch (IOException ioe) {
    System.err.println("Error de E/S leyendo " + fichero.toAbsolutePath() + ": " + ioe.getMessage());
    return;
}
```

Paso 4 — Convertir a hexadecimal la preview

Función simple para imprimir AA BB CC …

```
private static String aHex(byte[] data, int len) {
    if (len <= 0) return "(sin datos)";
    StringBuilder sb = new StringBuilder(len * 3);
    for (int i = 0; i < len; i++) {
        int v = data[i] & 0xFF;           // byte con signo → 0..255
        sb.append(String.format("%02X", v)); // 2 dígitos hex
        if (i < len - 1) sb.append(' ');
    }
    return sb.toString();
}
```

Paso 5 — Mostrar resultados

Al final del main (tras leer)

```
System.out.println("Fichero : " + fichero.toAbsolutePath());
System.out.println("Tamaño  : " + total + " bytes");
System.out.println("Preview : " + aHex(preview, previewLen));
```

Resultado:

```
Fichero : C:\...\flujo_bytes\listado.pdf
Tamaño  : 183421 bytes
Preview : 25 50 44 46 2D 31 2E 37 0A 25 E2 E3 CF D3 0A 0A
```

Ejercicio paso a paso. Contar bytes de un fichero

0) Objetivo

Crear un programa Java que **lea un fichero binario** (PDF, JPG, MP4…) **en bloques** y **cuente** cuántos bytes tiene.Se ejecutará por conso la con:

```
java flujo_bytes.ContarBytes <ruta-fichero> [--buf=N]
```

- `<ruta-fichero>`: fichero a medir.

- `--buf=N (opcional): **tamaño del bloque de lectura en bytes** (por defecto 8192 = 8 KiB).`

---

1) Preparar el proyecto y paquete

1. Crea una carpeta raíz del ejercicio (p. ej. Ejercicios).
2. Dentro, una carpeta flujo_bytes.
3. Archivo flujo_bytes/ContarBytes.java con la primera línea:

package flujo_bytes;

Por qué paquete: así practicamos la ejecución "real" con java paquete.Clase y evitamos nombres duplicados en proyectos grandes.

---

2) Leer los argumentos de entrada

Primero, solo imprimimos los argumentos para entender cómo llegan:

```
package flujo_bytes;

public class ContarBytes {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Uso: java flujo_bytes.ContarBytes <fichero>
[--buf=N]");
            return;
        }
        System.out.println("Argumento 1 (fichero): " + args[0]);
        if (args.length > 1) {
            System.out.println("Argumento 2 (opcional): " + args[1]);
        }
    }
}
```

Prueba (Windows, desde la carpeta raíz "Ejercicios"):

```
javac flujo_bytes\ContarBytes.java
java flujo_bytes.ContarBytes flujo_bytes\listado.pdf --buf=4096
```

---

3) ¿Qué es el bloque (buffer) y por qué importa?

- Cuando lees un archivo **byte a byte** harías millones de llamadas al SO → muy lento.
- Por eso leemos en **bloques (buffers)**: un byte[] de tamaño fijo, p. ej. **8192 bytes = 8 KiB**.

- **El tamaño del buffer NO cambia el resultado** (el conteo de bytes), **solo el rendimiento**.
- Un buffer demasiado **pequeño** ⇒ muchas lecturas/escrituras ⇒ más lento. Uno demasiado **grande** ⇒ más memoria y **no siempre** más rápido.

8192 (8 KiB) es un valor estándar en Java (equilibrio entre rendimiento y RAM). Aun así, conviene dejarlo **configurable** para que el alumno experimente.

Vamos a permitir cambiar el buffer con --buf=N y, si no viene, usar por defecto **8 KiB**.

Varios formatos:

Formato	**Ejemplo**	**Significa**
--buf=8192	--buf=8192	8192 bytes (8 KiB)
--buf=8k / --buf=8K	--buf=8k	8 × 1024 = 8192 bytes
--buf=1m / --buf=1M	--buf=1m	1 × 1024 × 1024 = 1 MiB
número “a pelo”	16384	16384 bytes (16 KiB)

Ejemplos de ejecución:

```
java flujo_bytes.ContarBytes flujo_bytes\listado.pdf
java flujo_bytes.ContarBytes flujo_bytes\listado.pdf --buf=4096
java flujo_bytes.ContarBytes flujo_bytes\listado.pdf --buf=64k
java flujo_bytes.ContarBytes flujo_bytes\listado.pdf --buf=1m
java flujo_bytes.ContarBytes flujo_bytes\listado.pdf 16384
```

---

4) Validar la ruta del fichero

Antes de leer, comprobamos:

- Que **existe**.
- Que es un **fichero** (no una carpeta).
- Que tenemos **permiso de lectura**.

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;

Path fichero = Paths.get(args[0]);
```

```
if (!Files.exists(fichero)) { /* error */ }
if (!Files.isRegularFile(fichero)) { /* error */ }
if (!Files.isReadable(fichero)) { /* error */ }
```

---

5) Leer en bloques con flujos de bytes

- Usamos FileInputStream envuelto en BufferedInputStream.
- Creamos un byte[] buffer = new byte[N].
- En un while, hacemos in.read(buffer):
    - Devuelve cuántos bytes se han leído (o -1 si EOF).
    - **Sumamos** al contador.

Idea del bucle:

```
long total = 0;
int leidos;
while ((leidos = in.read(buffer)) != -1) {
    total += leidos;
}
```

try-with-resources: el flujo se cierra solo al acabar el try.

---

6) Compilar y ejecutar correctamente (con paquete)

Desde la **carpeta raíz** Ejercicios:

```
javac flujo_bytes\ContarBytes.java
java flujo_bytes.ContarBytes flujo_bytes\listado.pdf
java flujo_bytes.ContarBytes flujo_bytes\listado.pdf --buf=16384
```

Si la ruta tiene **espacios**, usa **comillas**:

```
java flujo_bytes.ContarBytes "flujo_bytes\mi listado.pdf" --buf=8192
```

---

7) Comprueba que funciona

- Compara el total con lo que dice el Explorador (Propiedades → Tamaño).
- Prueba con otro --buf (4096, 8192, 65536): el número **no cambia**, solo el **rendimiento**.

---

8) Errores típicos y cómo solucionarlos

- **NoSuchFileException** → la ruta no es correcta (ojo con las comillas si hay espacios).

- **Compila bien pero no ejecuta** → asegúrate de ejecutar con **paquete**: java flujo_bytes.ContarBytes ….
- **Diferencias de versión** (UnsupportedClassVersionError): compilas con JDK 21 y ejecutas con Java 8.
  Solución rápida: javac --release 8 … o ejecuta con el Java del JDK 21.

Ejercicio paso a paso. Copia básica: sin buffer vs con buffer

Objetivo: copiar un fichero binario y medir el tiempo con y sin Buffered.
Logro: imprime tiempo (ms) de ambas versiones y compara.
Pistas:

- Sin buffer: FileInputStream/FileOutputStream con un byte[] pequeño (p. ej. 512).
- Con buffer: envolver en BufferedInputStream/BufferedOutputStream y byte[] de 8 KiB.
- Tiempo: System.nanoTime() o Instant/Duration.

Paso 0 — Estructura y "Hola mundo"

1. Crea la clase y método main:

```
package flujo_bytes;

public class CopiaComparativa {
   public static void main(String[] args) {
      System.out.println("CopiaComparativa listo.");
   }
}
```

2. Compila y ejecuta desde la **raíz del paquete** (tu carpeta "Ejercicios"):

```
javac flujo_bytes\CopiaComparativa.java
java flujo_bytes.CopiaComparativa
```

Paso 1 — Argumentos de entrada

Queremos:

```
java flujo_bytes.CopiaComparativa <origen> <dest_sin> <dest_con>
<origen>: fichero a copiar.
<dest_sin>: copia "sin buffer".
<dest_con>: copia "con buffer".
```

Añade validación mínima:

```
if (args.length < 3) {
   System.out.println("Uso: java flujo_bytes.CopiaComparativa <origen> <dest_sin> <dest_con>");
   return;
}
System.out.println("Origen   : " + args[0]);
System.out.println("Destino1 : " + args[1]);
System.out.println("Destino2 : " + args[2]);
```

Comprueba que imprime bien con rutas reales (usa comillas si hay espacios).

Paso 2 — Validar el origen

Antes de copiar: comprobar que **existe**, es **fichero** y es **legible**.

```
import java.nio.file.*;

private static void validarOrigen(Path origen) {
  if (!Files.exists(origen))
    throw new IllegalArgumentException("No existe: " + origen.toAbsolutePath());
  if (!Files.isRegularFile(origen))
    throw new IllegalArgumentException("No es fichero regular: " + origen.toAbsolutePath());
  if (!Files.isReadable(origen))
    throw new IllegalArgumentException("Sin permisos de lectura: " + origen.toAbsolutePath());
}
```

En main:

```
Path pOrigen = Paths.get(args[0]);
Path pSin    = Paths.get(args[1]);
Path pCon    = Paths.get(args[2]);

try {
  validarOrigen(pOrigen);
} catch (IllegalArgumentException iae) {
  System.err.println("Parámetros inválidos: " + iae.getMessage());
  return;
}
```

Paso 3 — Preparar destinos

Crea la carpeta padre si no existe. No borres aún nada.

```
private static void prepararDestino(Path destino) throws java.io.IOException {
  Path parent = destino.getParent();
  if (parent != null && !Files.exists(parent)) {
    Files.createDirectories(parent);
  }
}
```

En main:

```
try {
  prepararDestino(pSin);
  prepararDestino(pCon);
} catch (java.io.IOException ioe) {
  System.err.println("Error preparando destino: " + ioe.getMessage());
  return;
}
```

Paso 4 — Copia “sin buffer”

- Usa FileInputStream y FileOutputStream.
- **Bloque pequeño** (p. ej., 512 bytes) para notar diferencia.

- Cuenta bytes copiados.

```
import java.io.*;

private static long copiarSinBuffer(Path origen, Path destino, int bloque) throws IOException {
  long total = 0;
  byte[] buf = new byte[bloque];
  try (FileInputStream in = new FileInputStream(origen.toFile());
     FileOutputStream out = new FileOutputStream(destino.toFile(), false)) {
    int n;
    while ((n = in.read(buf)) != -1) {
      out.write(buf, 0, n);
      total += n;
    }
    out.flush();
  }
  return total;
}
```

En main, llama y prueba con un PDF/JPG pequeño:

```
int BLOCK_SMALL = 512;
long bytes1 = copiarSinBuffer(pOrigen, pSin, BLOCK_SMALL);
System.out.println("Sin buffer copiados: " + bytes1 + " bytes");
```

Paso 5 — Medir tiempo y MB/s

- Marca inicio/fin con Instant.now() y calcula Duration.
- Convierte bytes a MB y divide entre segundos.

```
import java.time.*;

private static void imprimirResumen(String titulo, long bytes, Duration dur) {
  double seg = Math.max(0.001, dur.toMillis() / 1000.0);
  double mb  = bytes / (1024.0 * 1024.0);
  double mbps = mb / seg;
  System.out.printf("%s → %.2f MB en %.2f s (%.2f MB/s)%n", titulo, mb, seg, mbps);
}
```

Úsalo con la copia sin buffer:

```
Instant t0 = Instant.now();
long bytes1 = copiarSinBuffer(pOrigen, pSin, BLOCK_SMALL);
Duration d1 = Duration.between(t0, Instant.now());
imprimirResumen("SIN BUFFER", bytes1, d1);
```

Paso 6 — Copia "con buffer"

- Envuelve con BufferedInputStream y BufferedOutputStream.
- **Buffer mayor** (8 KiB = 8192).

```
private static long copiarConBuffer(Path origen, Path destino, int bufferSize) throws IOException {
  long total = 0;
  byte[] buf = new byte[bufferSize];
  try (BufferedInputStream in  = new BufferedInputStream(new FileInputStream(origen.toFile()),
bufferSize);
```

```
      BufferedOutputStream out = new BufferedOutputStream(new
FileOutputStream(destino.toFile(), false), bufferSize)) {
      int n;
      while ((n = in.read(buf)) != -1) {
        out.write(buf, 0, n);
        total += n;
      }
      out.flush();
    }
    return total;
}
```

En main:

```
int BUFFER_BIG = 8 * 1024;
Instant t2 = Instant.now();
long bytes2 = copiarConBuffer(pOrigen, pCon, BUFFER_BIG);
Duration d2 = Duration.between(t2, Instant.now());
imprimirResumen("CON BUFFER", bytes2, d2);
```

Esperado: mismos bytes copiados, **mejor MB/s** con buffer.

Paso 7 — Flags opcionales (tamaños)

Permite ajustar tamaños desde línea de comandos:

- --block=512 para la copia sin buffer.
- --buf=8192 para la copia con buffer.

```
private static int parsePositiveIntOrDefault(String s, int defVal) {
    try {
      int v = Integer.parseInt(s.trim());
      return v > 0 ? v : defVal;
    } catch (NumberFormatException e) {
      return defVal;
    }
}
```

Parsea en main:

```
int BLOCK_SMALL = 512;
int BUFFER_BIG  = 8 * 1024;

for (int i = 3; i < args.length; i++) {
    String a = args[i] == null ? "" : args[i].trim();
    if (a.startsWith("--block=")) {
      BLOCK_SMALL = parsePositiveIntOrDefault(a.substring("--block=".length()), BLOCK_SMALL);
    } else if (a.startsWith("--buf=")) {
      BUFFER_BIG = parsePositiveIntOrDefault(a.substring("--buf=".length()), BUFFER_BIG);
    }
}
```

Paso 8 — Verificación opcional (CRC32)

Para asegurarte de que las copias son idénticas al original:

```
import java.util.zip.CRC32;

private static String crc32(Path fichero) throws IOException {
   CRC32 crc = new CRC32();
   byte[] buffer = new byte[8 * 1024];
   try (BufferedInputStream in = new BufferedInputStream(new FileInputStream(fichero.toFile()))) {
      int n;
      while ((n = in.read(buffer)) != -1) {
         crc.update(buffer, 0, n);
      }
   }
   return String.format("%08X", crc.getValue());
}
```

Añade flag --crc:

```
boolean doCRC = false;
for (int i = 3; i < args.length; i++) {
   if ("--crc".equalsIgnoreCase(args[i])) doCRC = true;
}

if (doCRC) {
   String crcO = crc32(pOrigen);
   String crcA = crc32(pSin);
   String crcB = crc32(pCon);
   System.out.println("\nCRC32 (hex):");
   System.out.println("  Origen   : " + crcO);
   System.out.println("  Sin buf  : " + crcA + (crcO.equals(crcA) ? " (OK)" : " (MISMATCH)"));
   System.out.println("  Con buf  : " + crcB + (crcO.equals(crcB) ? " (OK)" : " (MISMATCH)"));
}
```

Paso 9 — Compilar y ejecutar (recordatorio)

Desde la **raíz del paquete**:

```
javac flujo_bytes\CopiaComparativa.java
java flujo_bytes.CopiaComparativa flujo_bytes\listado.pdf flujo_bytes\listado_sinbuf.pdf
flujo_bytes\listado_conbuf.pdf --block=512 --buf=8192 --crc
```

Si hay espacios en rutas, usa **comillas**.

Si te sale **UnsupportedClassVersionError (65.0 vs 52.0)**, estás compilando con JDK 21 y ejecutando con Java 8. Soluciones:

- Ejecuta con Java 21 (ajusta PATH), o
- Compila con javac --release 8 … y ejecuta con Java 8.